Assignment 1 - Fully Connected Neural Networks

Group 15

February 08, 2026

Abstract

This report studies the behavior and limitations of Fully Connected Neural Networks (FCNNs) through three focused experiments. First, we implement an FCNN from scratch using NumPy on the UCI Adult dataset to study the effects of feature scaling and optimizer choice on convergence. Second, we analyze feature learning on MNIST using first-layer weight visualization and pixel scrambling to test whether FCNNs encode spatial structure. Third, we examine training stability in deep networks on Tiny ImageNet through vanishing gradient analysis and ablation experiments. The results show that FCNNs learn interpretable features but remain insensitive to spatial relationships in image data. We also find that ReLU activations, batch normalization, and adaptive optimizers are necessary for stable training in deep fully connected models.

1 The "No-Framework" Challenge (NumPy Only)

1.1 Problem Definition & Methodology

We implemented a 3-layer FCNN to predict income classes ($> 50K$ vs $\leq 50K$) using the UCI Adult Census Income dataset. The implementation relied solely on `NumPy` to understand the mathematical "engine" of deep learning.

Architecture:

- **Input Layer:** Matches feature dimension.
- **Hidden Layers:** Two layers (128 and 64 neurons) with **ReLU** activation.
- **Output Layer:** 1 neuron with **Sigmoid** activation for binary classification.
- **Initialization:** He-Initialization ($W \sim \mathcal{N}(0, \sqrt{2/n_{in}})$) was used to prevent vanishing/exploding gradients.

Optimization: We implemented both Stochastic Gradient Descent (SGD) and Adam optimization manually. The loss function used was Binary Cross-Entropy.

1.2 Experimental Results: The Importance of Pre-Processing

We compared the performance of the model using scaled vs. raw data and different optimizers.

Data Type	Optimizer	Test Accuracy	Observations
Scaled Data	SGD	**81.62%**	Smooth convergence, efficient learning.
Raw Data	SGD	**75.50%**	Stagnated at baseline; gradients exploded/vanished.
Raw Data	Adam	**84.72%**	Adaptive moments handled unscaled features effectively.

Table 1: Comparison of Model Performance on UCI Adult Dataset

Interpretation: Standard SGD fails on raw data because feature magnitudes vary wildly (e.g., Age vs. Capital Gain), causing the loss landscape to be elongated. Adam overcomes this by adapting learning rates per parameter.

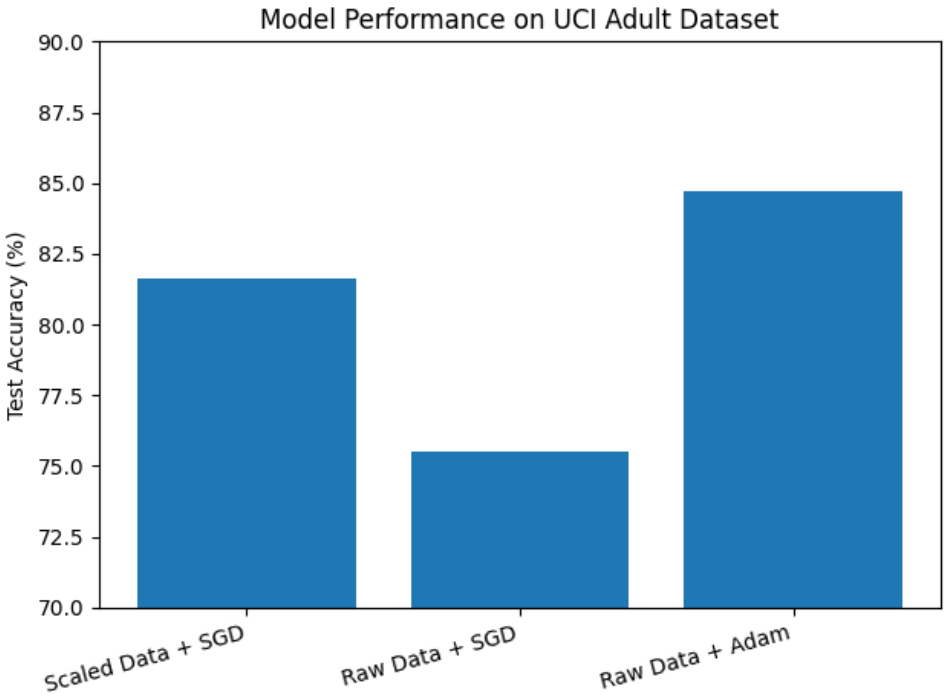

Figure 1: Test accuracy comparison for different data preprocessing and optimizer choices on the UCI Adult dataset.

2 Vision & Feature Interpretation

2.1 Weight Visualization (Latent Space Analysis)

Task: We studied what the network learns by visualizing the weights of the first hidden layer in a Fully Connected Neural Network (FCNN) trained on MNIST. The goal was to understand which visual patterns activate individual neurons.

Architecture:

- **Input Layer:** 784-dimensional flattened grayscale image.
- **Hidden Layer 1:** 512 neurons with ReLU activation.
- **Hidden Layer 2:** 256 neurons with ReLU activation.
- **Output layer:** 10 neurons, trained using Cross-Entropy loss.
- **Initialization:** He (Kaiming) initialization for all linear layers.
- **Regularization:** Dropout applied after each ReLU activation.

Method: After training, we extracted the weight matrix between the input layer and the first hidden layer (512×784). We reshaped each neuron's weight vector into a 28×28 image and visualized it as a heatmap.

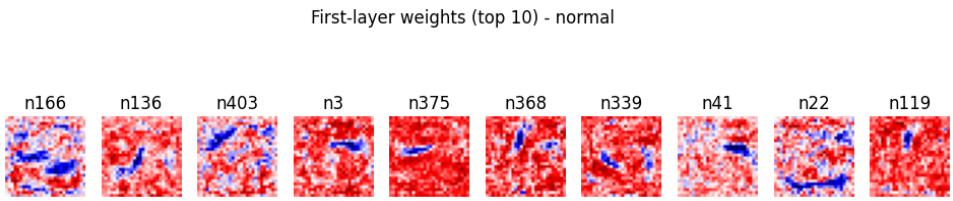

Figure 2: First-layer weights learned from normal MNIST inputs.

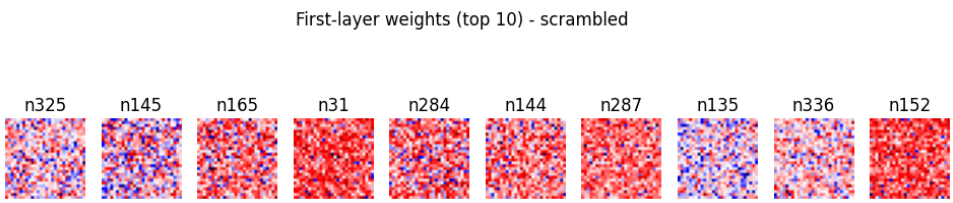

Figure 3: First-layer weights learned from scrambled MNIST inputs.

Observation: Many neurons show clear structure instead of random noise. Some weights form strokes, edges, or curved shapes similar to parts of handwritten digits. Other neurons respond to broader intensity regions.

Interpretation: Even with flattened inputs, the FCNN learns useful visual features. Neurons in the first hidden layer behave like simple pattern detectors. Each neuron responds to specific pixel combinations linked to digit shapes. This shows that FCNNs can learn interpretable features without explicit spatial modeling.

2.2 The "Flattening" Experiment

Task: We tested whether the FCNN depends on spatial relationships between pixels. We trained the model on a scrambled version of MNIST, where pixel positions are randomly permuted using a fixed permutation shared across all images.

Architecture: Both experiments used the same FCNN

- $784 \rightarrow 512 \rightarrow 256 \rightarrow 10$
- ReLU activation.
- He initialization
- Same optimizer, learning rate, and training schedule

Experimental Setup:

- **Normal MNIST:** Standard flattened pixel order.
- **Scrambled MNIST:** Flattened images with a fixed random pixel permutation
- **Training:** Models trained separately under identical settings

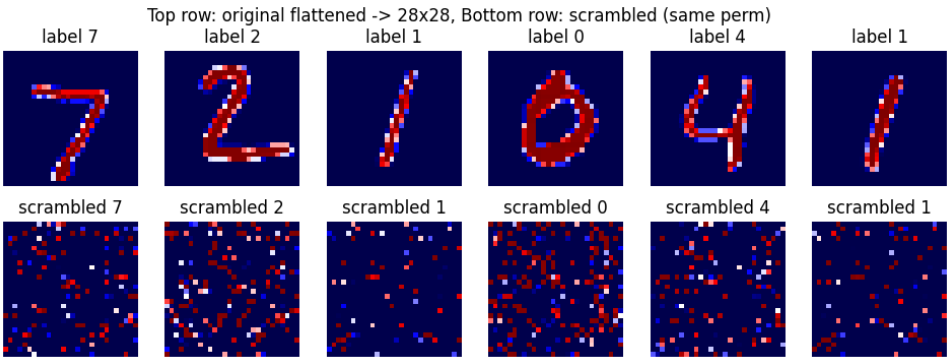

Figure 4: Original MNIST digits (top) and the same images after fixed pixel scrambling (bottom).

Observation: We tested whether the FCNN depends on spatial relationships between pixels. We trained the model on a scrambled version of MNIST, where pixel positions are randomly permuted using a fixed permutation shared across all images.

Interpretation: The FCNN does not use spatial structure. Each pixel connects independently to all neurons in the next layer, so the network treats the input as an unordered feature vector. When the permutation stays fixed, the model learns the new representation without difficulty. This highlights a limitation of FCNNs: they do not encode spatial locality. Spatial structure must be added through architectures such as convolutional networks.

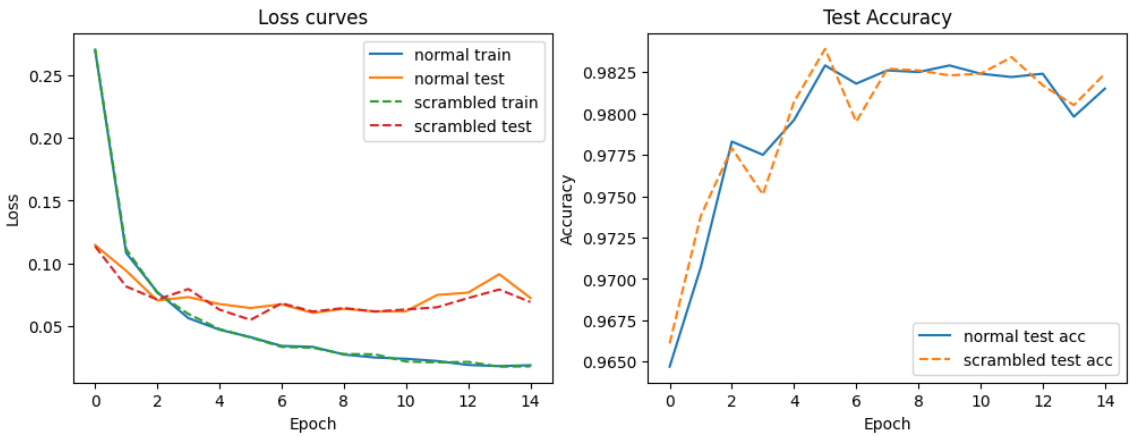

Figure 5: Training loss and test accuracy for normal and scrambled MNIST.

Dataset Condition	**Test Accuracy**
Normal MNIST	98.15%
Scrambled MNIST	98.24%

Table 2: Impact of pixel scrambling on FCNN performance

3 Stress Testing and Robustness

3.1 Vanishing Gradients & Modern Fixes

Task: We constructed a **very deep FCNN (8 layers)** and trained it on Tiny ImageNet-10 under two settings: Sigmoid activations throughout (no BatchNorm) & ReLU activations with Batch Normalization.

Architecture:

- **Input Layer:** Flattened RGB image of dimension $3 \times 64 \times 64$.
- **Hidden Layers:** A stack of 8 fully connected layers, each with 512 neurons.
- **Activation:** Sigmoid (Experiment A) or ReLU (Experiment B).
- **Batch Normalization:** Applied after each hidden layer only in Experiment B.
- **Dropout:** Optional dropout applied after activation (as per ablation setting).
- **Output Layer:** Fully connected layer with 10 neurons followed by Softmax (via Cross-Entropy loss) for multi-class classification.

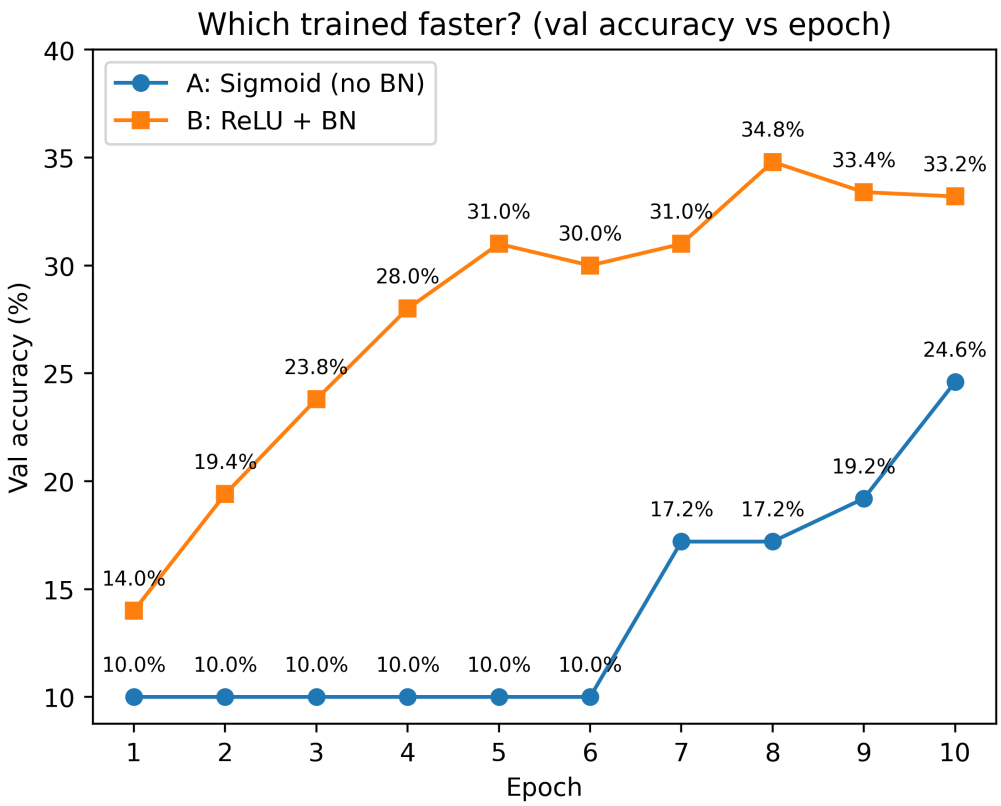

Figure 6: Validation accuracy vs epoch for Exp A (Sigmoid, no BN) and Exp B (ReLU + BN).

Observation: Experiment B (ReLU+BN) trains faster & reaches higher validation accuracy throughout training. It improves immediately from the first epoch & reaches $\sim 34.8\%$ around epoch 8.

Why Exp-B faster? The difference is explained by the first-layer gradient norms (Fig. 3, Table 3).

Epoch	Grad Norm (A: Sigmoid no BN)	Grad Norm (B: ReLU + BN)
1	0.0000054	1.7127753
2	0.0000026	0.3814512
3	0.0000021	0.2905418
4	0.0000019	0.2492541
5	0.0000029	0.2546917
6	0.0000173	0.2534654
7	0.0004075	0.2782212
8	0.0497779	0.2932410
9	0.1980109	0.3177722
10	0.1119177	0.3283682

Table 3: First-layer gradient norms per epoch for Experiment A vs Experiment B.

- In Exp A, the first-layer gradient norm is extremely small in early epochs ($\approx 10^{-6}$), meaning the first layer (or earlier layers) receives no effective update, so learning stalls (vanishing gradients due to Sigmoid saturation).
- In Exp B, gradient norms remain stable and non-trivial (~ 0.25 to 1.7), enabling consistent updates and faster learning. ReLU avoids saturation for positive activations, and BatchNorm keeps layer inputs well-scaled, preventing gradients from collapsing.

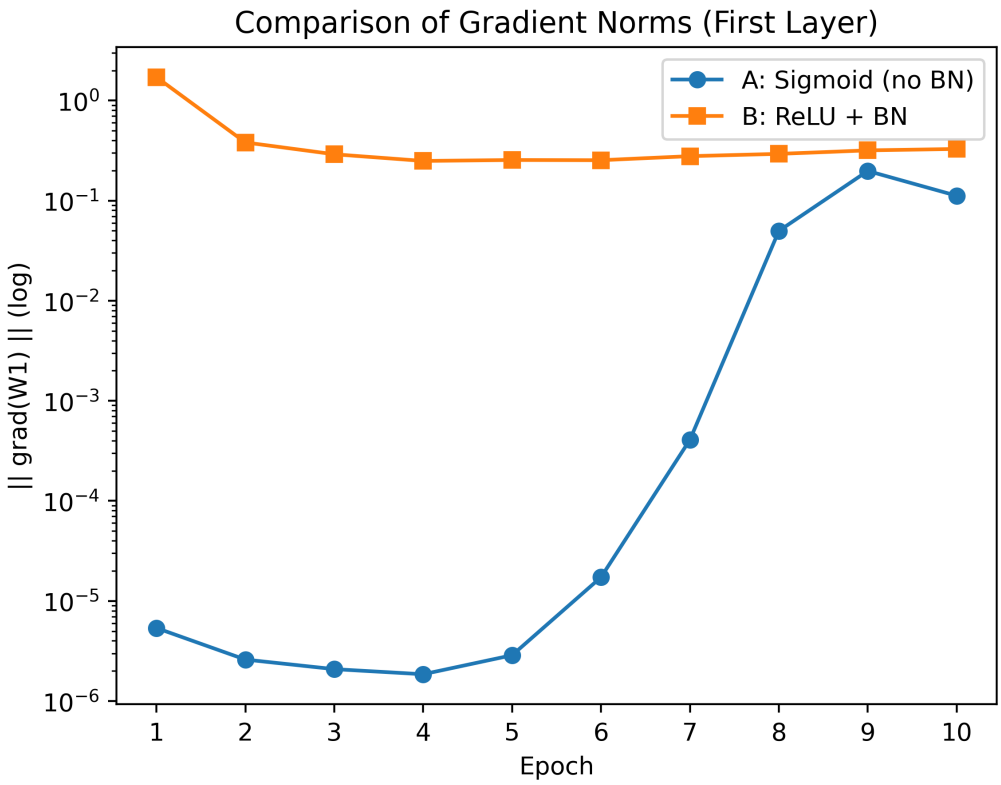

Figure 7: First-layer gradient norm $\|\nabla_{W_1}\|$ (log scale) for Exp A vs Exp B.

3.2 The Ablation Study (Switch-Off Test)

Task: We measured validation accuracy changes relative to the base setup by varying dropout, learning rate, and optimizer, showing these hyperparameters strongly affect performance and require tuning.

experiment	activation	batch_norm	dropout	LR	optimizer	val_acc	acc_diff vs base
Base	relu	True	0.5	0.001	adam	37.4%	0.0
No Dropout	relu	True	0.0	0.001	adam	43.8%	+6.4
LR x10	relu	True	0.5	0.01	adam	35.8%	-1.6
LR /10	relu	True	0.5	0.0001	adam	10.6%	-26.8
Using SGD	relu	True	0.5	0.001	sgd	10.2%	-27.2

Table 4: Showing which factor had the biggest impact on performance (switch-off test).

Key Findings:

1. **Dropout**: Removing dropout improved accuracy (+6.4 %) for this specific setting indicating that it caused underfitting rather than preventing overfitting.
2. **Optimizer:** Switching to SGD showed huge performance drop (−27.2 %), highlighting the necessity of adaptive learning rates for complex, unnormalized image data.
3. **Learning Rate:** The baseline LR of 0.001 was optimal. Increasing it caused mild instability (−1.6 %), while decreasing it slowed convergence significantly (−26.8 %).

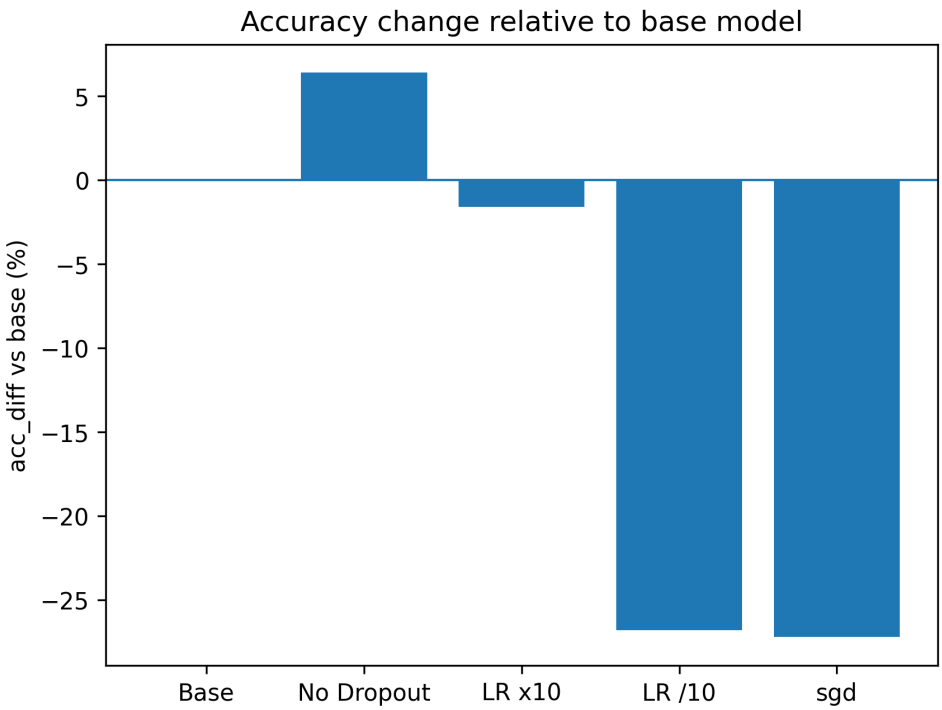

Figure 8: Accuracy change relative to the base model (%).